```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

1 этап) закрузка и первичный просмотр данных

```
user = pd.read_excel("dannye-dlia-keisa-po-pa-cebbefc3-5b29-4f88-b6d4-3336c88d743f.xlsx", sheet_name="Пользователи")
payment = pd.read_excel("dannye-dlia-keisa-po-pa-cebbefc3-5b29-4f88-b6d4-3336c88d743f.xlsx", sheet_name="Платежи")

user.head(20)
```

	user_id	group	age	city	device_type
0	1001	A	31	Москва	ios
1	1002	A	31	Москва	ios
2	1003	A	25	Москва	android
3	1004	A	20	Москва	ios
4	1005	A	34	Москва	ios
5	1006	A	42	Регион	android
6	1007	A	34	Регион	ios
7	1008	A	43	Регион	ios
8	1009	A	30	Москва	android
9	1010	A	26	Москва	android
10	1011	A	45	СПб	ios
11	1012	A	41	Регион	ios
12	1013	A	40	СПб	android
13	1014	A	31	СПб	ios
14	1015	A	38	Москва	ios
15	1016	A	34	Регион	ios
16	1017	A	26	Регион	ios
17	1018	A	39	Москва	ios
18	1019	A	27	Регион	ios
19	1020	A	22	СПб	ios

```
user["city"].value_counts()
```

```
city
Москва    595
Регион    514
СПб       394
Name: count, dtype: int64
```

```
user.device_type.value_counts()
```

```
device_type
android    777
ios        727
Name: count, dtype: int64
```

```
payment.head(20)
```

```
    payment_id  user_id group        step1_opened       step2_entered
\
0         5001   1001.0     A 2024-04-16 06:06:00 2024-04-16 06:10:00

1         5002   1002.0     A 2024-04-12 07:38:00 2024-04-12 07:41:00

2         5003   1003.0     A 2024-04-26 20:03:00 2024-04-26 20:07:00

3         5004   1004.0     A 2024-04-09 16:02:00                 NaT

4         5005   1004.0     A 2024-04-04 03:46:00 2024-04-04 03:54:00

5         5006   1004.0     A 2024-04-24 17:26:00 2024-04-24 17:27:00

6         5007   1005.0     A 2024-04-11 21:10:00 2024-04-11 21:11:00

7         5008   1006.0     A 2024-04-12 13:48:00 2024-04-12 13:52:00

8         5009   1007.0     A 2024-04-09 11:28:00 2024-04-09 11:30:00

9         5010   1007.0     A 2024-04-08 06:54:00                 NaT

10        5011   1007.0     A 2024-04-23 08:58:00 2024-04-23 08:59:00

11        5012   1008.0     A 2024-04-09 09:05:00                 NaT

12        5013   1009.0     A 2024-04-24 10:50:00 2024-04-24 10:54:00

13        5014   1009.0     A 2024-04-20 20:19:00 2024-04-20 20:20:00

14        5015   1010.0     A 2024-04-27 13:26:00                 NaT

15        5016   1011.0     A 2024-04-21 20:24:00 2024-04-21 20:25:00

16        5017   1011.0     A 2024-04-02 21:46:00 2024-04-02 21:48:00

17        5018   1011.0     A 2024-04-16 18:47:00 2024-04-16 18:48:00

18        5019   1012.0     A 2024-04-07 15:25:00 2024-04-07 15:26:00

19        5020   1013.0     A 2024-04-04 21:26:00 2024-04-04 21:31:00


       step3_confirmed       step4_success
0                  NaT                 NaT
1                  NaT                 NaT
2                  NaT                 NaT
3                  NaT                 NaT
4  2024-04-04 03:57:00 2024-04-04 04:02:00
```

```
5  2024-04-24 17:30:00 2024-04-24 17:36:00
6                  NaT                 NaT
7  2024-04-12 13:55:00 2024-04-12 14:00:00
8  2024-04-09 11:35:00 2024-04-09 11:42:00
9                  NaT                 NaT
10 2024-04-23 09:02:00                 NaT
11                 NaT                 NaT
12                 NaT                 NaT
13                 NaT                 NaT
14                 NaT                 NaT
15 2024-04-21 20:28:00 2024-04-21 20:33:00
16 2024-04-02 21:50:00 2024-04-02 21:56:00
17 2024-04-16 18:51:00 2024-04-16 18:57:00
18 2024-04-07 15:28:00 2024-04-07 15:33:00
19                 NaT                 NaT
```

```
user.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1504 entries, 0 to 1503
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   user_id      1504 non-null   int64 
 1   group        1503 non-null   str   
 2   age          1504 non-null   int64 
 3   city         1503 non-null   str   
 4   device_type  1504 non-null   str   
dtypes: int64(2), str(3)
memory usage: 58.9 KB
```

здесь можно увидеть что есть некий 1-2 пользователя у которых нет группы и города

перейдем к платежу

```
payment.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2661 entries, 0 to 2660
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype         
---  ------           --------------  -----         
 0   payment_id       2661 non-null   int64         
 1   user_id          2660 non-null   float64       
 2   group            2661 non-null   str           
 3   step1_opened     2661 non-null   datetime64[us]
 4   step2_entered    2256 non-null   datetime64[us]
 5   step3_confirmed  1719 non-null   datetime64[us]
 6   step4_success    1522 non-null   datetime64[us]
```

```
dtypes: datetime64[us](4), float64(1), int64(1), str(1)
memory usage: 145.7 KB
```

```
payment["group"].value_counts()
```

```
group
B    1352
A    1309
Name: count, dtype: int64
```

по платежам можно заметить что один платеж не привязан ни к одному айди, + воронка постепенно сужается. Также можно заметить что записей в платежах больше чем записей в пользователях. Это может означать что возможно есть пользователи которые несколько раз пытались оплатить жкх

```
user["age"].describe()
```

```
count    1504.000000
mean       32.745346
std         8.410915
min        -5.000000
25%        27.000000
50%        33.000000
75%        38.000000
max       150.000000
Name: age, dtype: float64
```

describe по платежам нам ничего не даст поскольку там даты поэтому не будет его делать

теперь необходимо отфильтровать датасет от аномальных значений, записей без принадлежности к группе А или Б, или платежи без айди тд.

```
new_user = user[(user["age"] >= 18) & (user["age"] <= 100) &
(~user["group"].isna()) & (~user["city"].isna())]
new_user["age"].describe()
```

```
count    1500.000000
mean       32.699333
std         7.795048
min        20.000000
25%        27.000000
50%        33.000000
75%        38.000000
max        60.000000
Name: age, dtype: float64
```

```
new_payment = payment[(~payment["user_id"].isna()) &
(~payment["group"].isna())]
new_payment.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2660 entries, 0 to 2659
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   payment_id       2660 non-null   int64
 1   user_id          2660 non-null   float64
 2   group            2660 non-null   str
 3   step1_opened     2660 non-null   datetime64[us]
 4   step2_entered    2256 non-null   datetime64[us]
 5   step3_confirmed  1719 non-null   datetime64[us]
 6   step4_success    1522 non-null   datetime64[us]
dtypes: datetime64[us](4), float64(1), int64(1), str(1)
memory usage: 145.6 KB
```

объединяем 2 страницы таблицы в 1 датасет для удобства

```
df = new_payment.merge(new_user, on = "user_id", how = "inner")
df.head(20)
```

```
    payment_id  user_id group_x        step1_opened
step2_entered  \
0         5001   1001.0       A 2024-04-16 06:06:00 2024-04-16
06:10:00
1         5002   1002.0       A 2024-04-12 07:38:00 2024-04-12
07:41:00
2         5003   1003.0       A 2024-04-26 20:03:00 2024-04-26
20:07:00
3         5004   1004.0       A 2024-04-09 16:02:00
NaT
4         5005   1004.0       A 2024-04-04 03:46:00 2024-04-04
03:54:00
5         5006   1004.0       A 2024-04-24 17:26:00 2024-04-24
17:27:00
6         5007   1005.0       A 2024-04-11 21:10:00 2024-04-11
21:11:00
7         5008   1006.0       A 2024-04-12 13:48:00 2024-04-12
13:52:00
8         5009   1007.0       A 2024-04-09 11:28:00 2024-04-09
11:30:00
9         5010   1007.0       A 2024-04-08 06:54:00
NaT
10        5011   1007.0       A 2024-04-23 08:58:00 2024-04-23
08:59:00
11        5012   1008.0       A 2024-04-09 09:05:00
NaT
12        5013   1009.0       A 2024-04-24 10:50:00 2024-04-24
10:54:00
13        5014   1009.0       A 2024-04-20 20:19:00 2024-04-20
```

```
20:20:00
14        5015   1010.0       A 2024-04-27 13:26:00
NaT
15        5016   1011.0       A 2024-04-21 20:24:00 2024-04-21
20:25:00
16        5017   1011.0       A 2024-04-02 21:46:00 2024-04-02
21:48:00
17        5018   1011.0       A 2024-04-16 18:47:00 2024-04-16
18:48:00
18        5019   1012.0       A 2024-04-07 15:25:00 2024-04-07
15:26:00
19        5020   1013.0       A 2024-04-04 21:26:00 2024-04-04
21:31:00

       step3_confirmed       step4_success group_y  age    city
device_type
0                  NaT                 NaT       A   31  Москва
ios
1                  NaT                 NaT       A   31  Москва
ios
2                  NaT                 NaT       A   25  Москва
android
3                  NaT                 NaT       A   20  Москва
ios
4  2024-04-04 03:57:00 2024-04-04 04:02:00       A   20  Москва
ios
5  2024-04-24 17:30:00 2024-04-24 17:36:00       A   20  Москва
ios
6                  NaT                 NaT       A   34  Москва
ios
7  2024-04-12 13:55:00 2024-04-12 14:00:00       A   42  Регион
android
8  2024-04-09 11:35:00 2024-04-09 11:42:00       A   34  Регион
ios
9                  NaT                 NaT       A   34  Регион
ios
10 2024-04-23 09:02:00                 NaT       A   34  Регион
ios
11                 NaT                 NaT       A   43  Регион
ios
12                 NaT                 NaT       A   30  Москва
android
13                 NaT                 NaT       A   30  Москва
android
14                 NaT                 NaT       A   26  Москва
android
15 2024-04-21 20:28:00 2024-04-21 20:33:00       A   45     СПб
ios
16 2024-04-02 21:50:00 2024-04-02 21:56:00       A   45     СПб
```

```
ios
17 2024-04-16 18:51:00 2024-04-16 18:57:00       A   45     СПб
ios
18 2024-04-07 15:28:00 2024-04-07 15:33:00       A   41  Регион
ios
19                 NaT                 NaT       A   40     СПб
android
```

уберем дублирующиеся колонки и переименуем для удобства

```
df = df.drop(columns = ["group_y"])

df = df.rename(columns = {"group_x": "group"})
```

Сделаем колонку "флаг" чтобы понять где получилось оплатить а где нет, будем использовать это далее для группировки по каждому пользователю, где groupby даст либо 1 либо 0

```
df["successed"] = ~df["step4_success"].isna()

grouped = df[["user_id", "group", "successed"]].groupby(by =
"user_id").agg({"successed": "max", "group": "first"})

print(grouped)
```

```
         successed group
user_id
1001.0       False     A
1002.0       False     A
1003.0       False     A
1004.0        True     A
1005.0        True     A
...            ...   ...
2496.0        True     B
2497.0       False     B
2498.0        True     B
2499.0        True     B
2500.0       False     B

[1500 rows x 2 columns]
```

здесь можно увидеть что у некоторых клиентов количество оплат превысило 1 раз за месяц, но нас пока что это не интересует, главная текущая задача узнать конверсию успешных платежей поголовно

```
new_df = grouped.reset_index()
A_conv = round(new_df[new_df["group"] == "A"]["successed"].mean(), 3)
B_conv = round(new_df[new_df["group"] != "A"]["successed"].mean(), 3)
print(A_conv, B_conv, sep = "\n")
```

```
0.657
0.804
```

как можно заметить по результатам конверсия группы Б выше чем у группы А. разница довольно высокая аж почти 15%. какова вероятность того, что разница в 15 процентов - это просто случайность?

На это я выдвигаю 2 гипотезы, где

h0 - разница в 15 процентов это случайное совпадение, а упрощение заполнения реквизитов ни на что не влияет

h1 - разница в 15 процентов не случайна, новая форма явно лучше работает

использовать мы будем Z-тест для сравнения полученных конверсий поскольку на это есть 3 причины:

1) бинарность нашей метрики (прошла оплата или нет)

2) группы А и В независимы друг от друга

3) размер выборки 2 групп достаточен(750 записей)

подготовим базовые цифры для нахождения z-score

```
# количество успехов в 2 группах
A_suc = new_df[new_df["group"] == "A"]["successed"].sum()
B_suc = new_df[new_df["group"] != "A"]["successed"].sum()


#общее кол-во пользователей
A_count = len(new_df[new_df["group"] == "A"])
B_count = len(new_df[new_df["group"] != "A"])



#расчет общей доли успехов без дифференциации по группам
suc = (A_suc + B_suc)/(A_count+B_count)



# расчитаем еще стандартное отклонение между 2 группами (через
дисперсию распределения Бернулли)
p = np.sqrt(suc * (1- suc) * (1/A_count + 1/B_count))


# рассчитываем z
z = (B_suc/B_count - A_suc/A_count)/p
print(z)
```

```
6.402393383329898
```

z	0	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
+0	.50000	.50399	.50798	.51197	.51595	.51994	.52392	.52790	.53188	.53586
+0.1	.53983	.54380	.54776	.55172	.55567	.55966	.56360	.56749	.57142	.57535
+0.2	.57926	.58317	.58706	.59095	.59483	.59871	.60257	.60642	.61026	.61409
+0.3	.61791	.62172	.62552	.62930	.63307	.63683	.64058	.64431	.64803	.65173
+0.4	.65542	.65910	.66276	.66640	.67003	.67364	.67724	.68082	.68439	.68793
+0.5	.69146	.69497	.69847	.70194	.70540	.70884	.71226	.71566	.71904	.72240
+0.6	.72575	.72907	.73237	.73565	.73891	.74215	.74537	.74857	.75175	.75490
+0.7	.75804	.76115	.76424	.76730	.77035	.77337	.77637	.77935	.78230	.78524
+0.8	.78814	.79103	.79389	.79673	.79955	.80234	.80511	.80785	.81057	.81327
+0.9	.81594	.81859	.82121	.82381	.82639	.82894	.83147	.83398	.83646	.83891
+1	.84134	.84375	.84614	.84849	.85083	.85314	.85543	.85769	.85993	.86214
+1.1	.86433	.86650	.86864	.87076	.87286	.87493	.87698	.87900	.88100	.88298
+1.2	.88493	.88686	.88877	.89065	.89251	.89435	.89617	.89796	.89973	.90147
+1.3	.90320	.90490	.90658	.90824	.90988	.91149	.91308	.91466	.91621	.91774
+1.4	.91924	.92073	.92220	.92364	.92507	.92647	.92785	.92922	.93056	.93189
+1.5	.93319	.93448	.93574	.93699	.93822	.93943	.94062	.94179	.94295	.94408
+1.6	.94520	.94630	.94738	.94845	.94950	.95053	.95154	.95254	.95352	.95449
+1.7	.95543	.95637	.95728	.95818	.95907	.95994	.96080	.96164	.96246	.96327
+1.8	.96407	.96485	.96562	.96638	.96712	.96784	.96856	.96926	.96995	.97062
+1.9	.97128	.97193	.97257	.97320	.97381	.97441	.97500	.97558	.97615	.97670
+2	.97725	.97778	.97831	.97882	.97932	.97982	.98030	.98077	.98124	.98169
+2.1	.98214	.98257	.98300	.98341	.98382	.98422	.98461	.98500	.98537	.98574
+2.2	.98610	.98645	.98679	.98713	.98745	.98778	.98809	.98840	.98870	.98899
+2.3	.98928	.98956	.98983	.99010	.99036	.99061	.99086	.99111	.99134	.99158
+2.4	.99180	.99202	.99224	.99245	.99266	.99286	.99305	.99324	.99343	.99361
+2.5	.99379	.99396	.99413	.99430	.99446	.99461	.99477	.99492	.99506	.99520
+2.6	.99534	.99547	.99560	.99573	.99585	.99598	.99609	.99621	.99632	.99643
+2.7	.99653	.99664	.99674	.99683	.99693	.99702	.99711	.99720	.99728	.99736
+2.8	.99744	.99752	.99760	.99767	.99774	.99781	.99788	.99795	.99801	.99807
+2.9	.99813	.99819	.99825	.99831	.99836	.99841	.99846	.99851	.99856	.99861
+3	.99865	.99869	.99874	.99878	.99882	.99886	.99889	.99893	.99896	.99900
+3.1	.99903	.99906	.99910	.99913	.99916	.99918	.99921	.99924	.99926	.99929
+3.2	.99931	.99934	.99936	.99938	.99940	.99942	.99944	.99946	.99948	.99950
+3.3	.99952	.99953	.99955	.99957	.99958	.99960	.99961	.99962	.99964	.99965
+3.4	.99966	.99968	.99969	.99970	.99971	.99972	.99973	.99974	.99975	.99976
+3.5	.99977	.99978	.99978	.99979	.99980	.99981	.99981	.99982	.99983	.99983
+3.6	.99984	.99985	.99985	.99986	.99986	.99987	.99987	.99988	.99988	.99989
+3.7	.99989	.99990	.99990	.99990	.99991	.99991	.99992	.99992	.99992	.99992
+3.8	.99993	.99993	.99993	.99994	.99994	.99994	.99994	.99995	.99995	.99995
+3.9	.99995	.99995	.99996	.99996	.99996	.99996	.99996	.99996	.99997	.99997
+4	.99997	.99997	.99997	.99997	.99997	.99997	.99998	.99998	.99998	.99998

По таблице можно понять что полученный z говорит что полученный нами результат в A/B тесте с вероятностью более 99,99% достоверен.

Перейдем к визуализации

```
fig = plt.figure(figsize = (15, 25))


gs = fig.add_gridspec(nrows = 2, ncols= 1)
```

```
ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[1, 0])



sns.barplot(data = new_df, x = "group", y = "successed", ax = ax1,
palette= "deep", errorbar=None)
ax1.set_yticks(np.arange(0, 1, 0.05))

#подготовим данные для визуализации через lineplot(буду
визуализировать 2 воронки у 2 групп)

class Step_count:
    def __init__(self, name, data):
        self.name = name
        self.data = data


    def count(self):
        self.step1 = self.data[(self.data["step1_opened"].notna()) &
(self.data["group"] == self.name)]["user_id"].nunique()
        self.step2 = self.data[(self.data["step2_entered"].notna()) &
(self.data["group"] == self.name)]["user_id"].nunique()
        self.step3 = self.data[(self.data["step3_confirmed"].notna())
& (self.data["group"] == self.name)]["user_id"].nunique()
        self.step4 = self.data[(self.data["step4_success"].notna()) &
(self.data["group"] == self.name)]["user_id"].nunique()
        data_for_dataframe = {"step": ["step1", "step2", "step3",
"step4"],"count": [self.step1, self.step2, self.step3, self.step4],
"group": None}
        self.result = pd.DataFrame(data_for_dataframe)
        self.result["group"] = self.name

#посчитаем для A и B

a = Step_count("A", df)
b = Step_count("B", df)


a.count()
b.count()

f =  pd.concat((a.result, b.result))

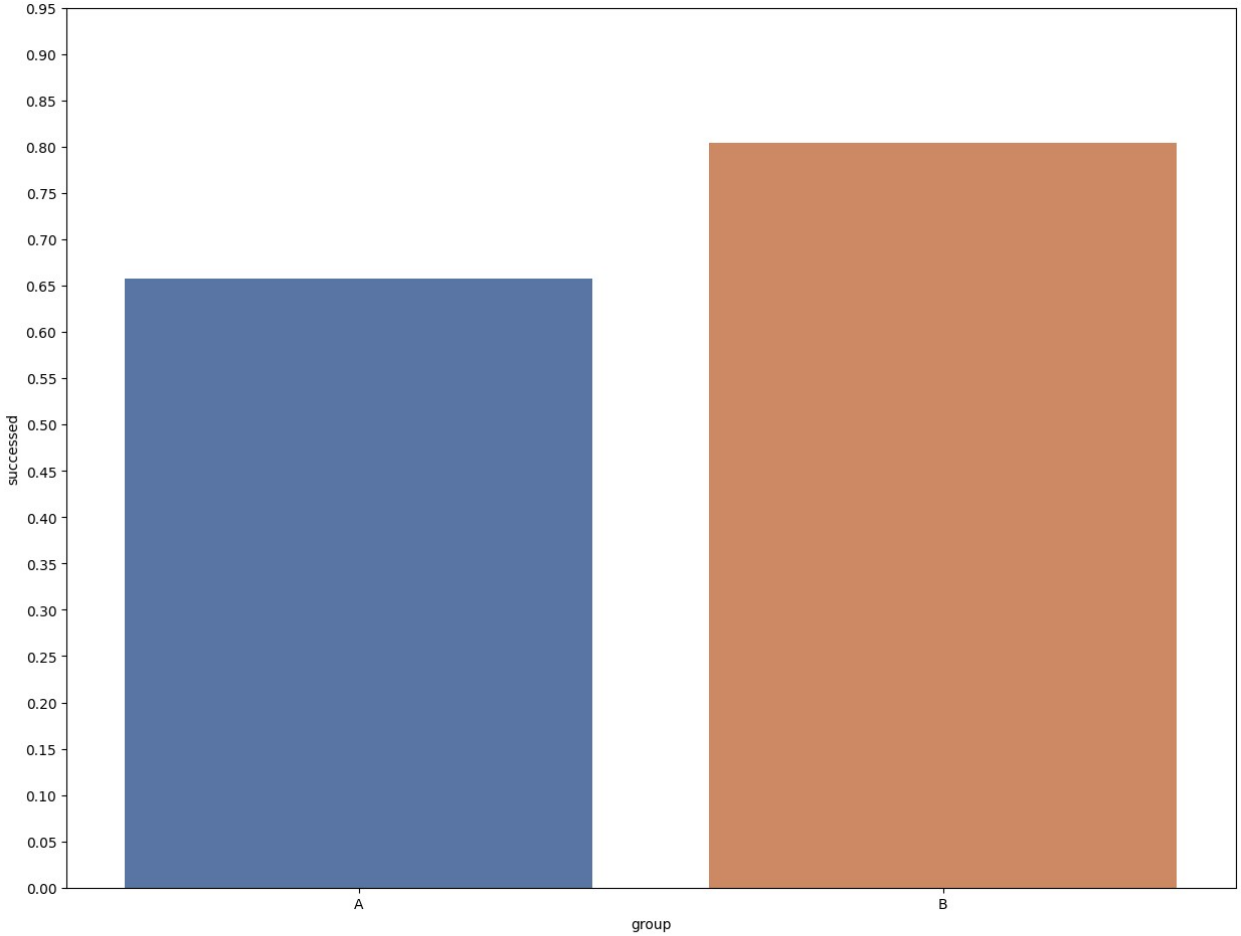

sns.lineplot(data = f, ax = ax2, x = f["step"], y = f["count"], hue =
"group", palette = "deep" )
```

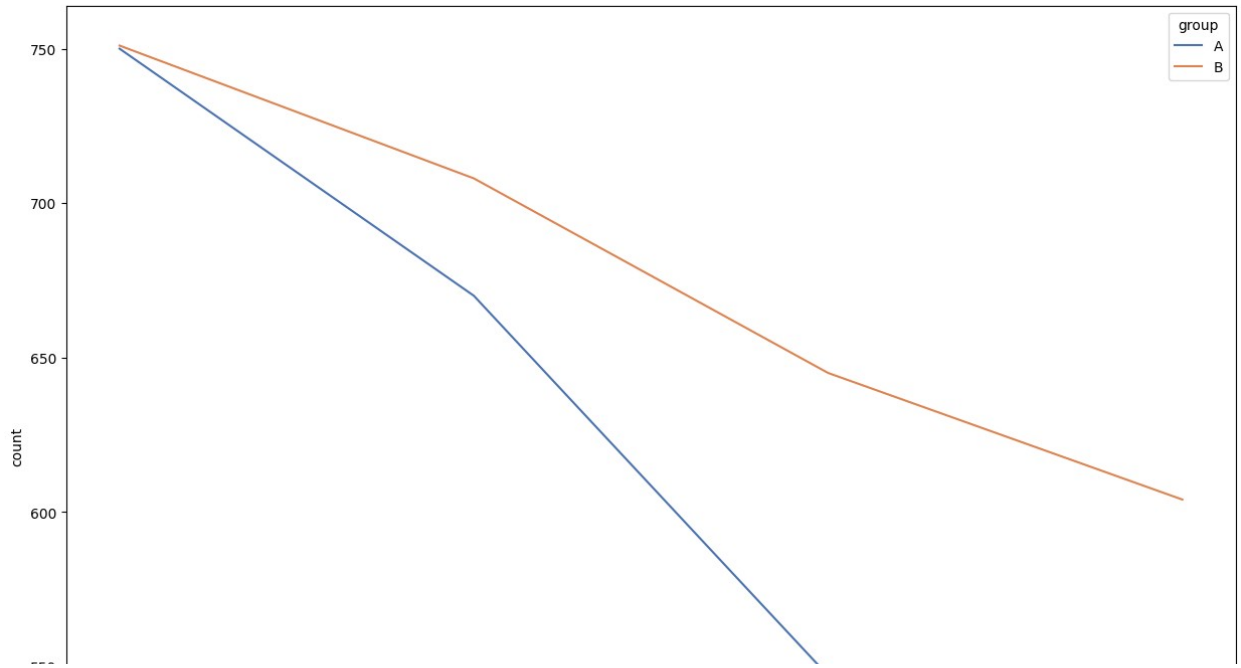
```
/var/folders/8m/fmz_cj0x64n4dh170vg54ggc0000gn/T/
ipykernel_28237/651445456.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.barplot(data = new_df, x = "group", y = "successed", ax = ax1,
palette= "deep", errorbar=None)

<Axes: xlabel='step', ylabel='count'>
```


successed
group
A
B


count
group
A
B